

# Exercise 11 — Understanding What to Change with AI

---

## Exercise Description

---

Use AI to analyse and improve three code examples (Java, Python, JavaScript), each targeting a different prompting strategy. No runnable code/tests required — compare solutions and document learnings and insights.

---

### Example 1 — Code Readability Improvement (Java, UserMgr )

---

**Issues identified:** - **Cryptic names:** class UserMgr, class U, fields u\_list, methods a(...) and f(...), params un/pw/em. - **Security:** the SQL is built by **string concatenation** of user input → SQL injection. This is the single most important change. - **Design:** the in-memory list is scanned linearly for duplicate usernames (O(n)); a Map<String,User> or a DB unique constraint is better. Passwords are stored/inserted in plaintext. - **Return semantics:** a(...) mixes validation failure and DB result in one boolean; callers can't tell *why* it failed.

#### Improved version (sketch):

```
class UserManager {
    private final Map<String, User> usersByName = new HashMap<>();
    private final DbConnection db;

    UserManager(DbConnection db) { this.db = db; }

    boolean register(String username, String password, String email) {
        if (!isValid(username, password, email)) return false;
        if (usersByName.containsKey(username)) return false;
        User user = new User(username, hash(password), email);
        usersByName.put(username, user);
        return db.execute(
            "INSERT INTO users (username, password, email) VALUES (?, ?, ?)",
            username, hash(password), email);    // parameterised – no injection
    }

    User findByUsername(String username) { return usersByName.get(username); }
}
```

**Insight I might have missed without AI:** the parameterised-query fix — it's easy to focus on renaming and overlook the injection vulnerability, which matters far more.

## Example 2 — Function Refactoring (Python, `process_orders`)

---

**One function does too much:** inventory validation, quantity check, customer check, pricing, premium discount, inventory mutation, shipping, tax, revenue accumulation, and result assembly — all in one loop.

**Decomposition plan (single-responsibility helpers):** - `validate_order(order, inventory, customer_data)` -> error | None - `price_order(order, inventory, customer_data)` -> (price, shipping, tax, final\_price) (with `apply_premium_discount`, `calculate_shipping`, `calculate_tax` as sub-helpers) - `reserve_inventory(inventory, item_id, quantity)` (the side effect, isolated) - `build_result(order, pricing)` - `process_orders(...)` becomes an orchestration loop that collects `results`, `error_orders`, and `total_revenue`.

**Benefits:** each piece is unit-testable; the side effect (inventory mutation) is isolated from the pure pricing math; magic numbers (0.9 discount, 0.08 tax, shipping tiers) move into named helpers/constants.

**Insight:** separating the **pure** pricing calculations from the **impure** inventory mutation is the highest-value split — it makes pricing trivially testable.

## Example 3 — Code Duplication Detection (JavaScript, `calculateUserStatistics`)

---

**Duplication:** three near-identical average loops (age/income/salary... actually age/income/score) and three near-identical "find highest" loops — six loops that differ only by field name.

**Consolidation:**

```
const mean = (arr, key) => arr.reduce((s, x) => s + x[key], 0) / arr.length;
const maxOf = (arr, key) => arr.reduce((m, x) => Math.max(m, x[key]), -Infinity);

function calculateUserStatistics(userData) {
  const stat = key => ({ average: mean(userData, key), highest: maxOf(userData, key) });
  return { age: stat('age'), income: stat('income'), score: stat('score') };
}
```

**For a junior team,** the helper functions (`mean`, `maxOf`) are more readable than a fully generic metaprogramming approach; adding a new metric becomes a one-line `stat('newField')`.

**Insight:** the win isn't just fewer lines — it's that a bug fix (e.g. empty-array handling) now happens in **one** place instead of six.

---

## Reflection Questions

---

1. **Which prompting strategy was most useful?** The **readability** prompt on the Java example — because it surfaced the SQL-injection risk I could easily have skimmed past while renaming.
  2. **What did the AI suggest that I might not have?** Parameterised queries (security), and isolating the impure inventory mutation from pure pricing.
  3. **Anything I disagreed with?** A fully generic "iterate all numeric keys" version of Example 3 — clever but harder for juniors to read than three explicit `stat()` calls; I'd keep it explicit.
  4. **How would I adapt these prompts for my stack?** Add house-style constraints (naming conventions, our ORM/query builder, our lint rules) so suggestions match the codebase.
  5. **Safeguards before applying to production?** Ensure tests exist and pass before and after; do security review on any query changes; refactor in small, reviewed commits.
- 

## Submission for Exercise 11 — Understanding What to Change with AI

---

1. **Example 1 (Java readability):** renamed `UserMgr/U/u_list/a/f` to intention-revealing names, **fixed SQL injection via parameterised query**, replaced linear duplicate scan with a map, and hashed passwords — the injection fix was the issue I'd most likely have missed.
2. **Example 2 (Python function refactoring):** decomposed `process_orders` into `validate_order`, `price_order` (+ discount/shipping/tax helpers), `reserve_inventory`, and `build_result`, with the orchestration loop left thin; key move is separating pure pricing from the impure inventory mutation.
3. **Example 3 (JS duplication):** collapsed six copy-paste loops into `mean/maxOf` helpers plus a `stat(key)` factory; the real benefit is single-point bug fixing, and I deliberately kept it explicit rather than fully generic for junior readability.
4. **Reflection answers:** provided above — most useful strategy was the readability prompt (found the injection), main disagreement was over-genericising Example 3, and the key safeguard is tests-green-before-and-after plus security review on query changes.